# Simulation Report

Topic:

Pipeline Processor Design and Simulation

Internship Task 3 – VLSI Design

Name: Likhith Gowda H R

Tool Used: Cadence Genus, SimVision

# TABLE OF CONTENTS


| Chapter No. | Title | Page No. |
|---|---|---|
| **1** | **INTRODUCTION** | **1–4** |
| 1.1 | Introduction | 1 |
| 1.2 | Theory of Pipeline Processor | 2 |
| 1.3 | Objective | 3 |
| 1.4 | Design Methodology | 4 |
| **2** | **DESIGN METHODOLOGY** | **5-7** |
| 2.1 | Pipeline Processor Design Implementation | 5 |
| 2.2 | Testbench Implementation | 6 |
| 2.3 | RTL / Schematic View | 7 |
| **3** | **SIMULATION** | **8–10** |
| **3.1** | Simulation Methodology | **8** |
| **3.2** | Simulation Results and Analysis | **9** |
| **4** | **SYNTHESIS AND PERFORMANCE ANALYSIS** | **11-14** |
| **4.1** | Power Analysis | **11** |
| **4.2** | Area Analysis | **12** |
| **4.3** | Timing Analysis | **13** |
| **4.4** | Gate-Level Report | **14** |
| **5** | **RESULTS** | **15** |
| **6** | **DISCUSSION** | **16** |
| **7** | **CONCLUSION** | **17** |

## LIST OF TABLES


| Table No. | Title | Page No. |
|---|---|---|
| 1.2.1 | Basic Instruction Set of the 4-Stage Pipelined Processor | 3 |


## LIST OF FIGURES


| Figure No. | Title | Page No. |
|---|---|---|
| 2.1.1 | Verilog Implementation of 4-Stage Pipelined Processor | 5 |
| 2.2.1 | Testbench Implementation of 4-Stage Pipelined Processor | 6 |
| 2.3.1 | Synthesized RTL Schematic of 4-Stage Pipelined Processor | 7 |
| 3.2.1 | Simulation Waveform of 4-Stage Pipelined Processor | 9 |
| 4.1.1 | Power Analysis Report | 11 |
| 4.2.1 | Area Utilization Report | 12 |
| 4.3.1 | Timing Analysis Report | 13 |
| 4.4.1 | Gate-Level Synthesis Report | 14 |

# Chapter 1
# Introduction

## 1.1 Introduction

Processors are the core components of digital systems that execute instructions and perform data processing operations. To improve processing speed and efficiency, modern processors use a technique called pipelining. Pipelining allows multiple instructions to be executed simultaneously by dividing instruction execution into several stages.

A Pipeline Processor is a processor architecture where instruction execution is separated into different stages such as instruction fetch, instruction decode, execution, and write-back. Each stage performs a specific task, and multiple instructions can be processed in different stages at the same time. This increases the overall throughput and performance of the processor.

In this project, a Pipeline Processor is designed using Verilog Hardware Description Language (HDL) and verified using Cadence simulation tools. The functional behavior of the processor is tested using a testbench, and simulation waveforms are analyzed using SimVision. The synthesized design is further analyzed for performance parameters such as area, power, and timing.

## 1.2 Theory of Pipeline Processor

A Pipeline Processor is a sequential digital system that executes instructions by dividing the instruction execution process into multiple stages. Each stage performs a specific part of the instruction processing, allowing several instructions to be processed simultaneously.

The main stages in a pipeline processor include:

- **Instruction Fetch (IF):** Fetches the instruction from instruction memory.
- **Instruction Decode (ID):** Decodes the instruction and identifies the required operands.
- **Execution (EX):** Performs arithmetic or logical operations.
- **Write Back (WB):** Stores the result into the register file.

Pipeline registers are used between each stage to store intermediate results and allow instructions to move through the pipeline in synchronization with the clock signal. Pipelining improves processor performance by increasing instruction throughput and reducing execution time.

The processor supports a set of basic instructions which are executed during the execution stage of the pipeline. These instructions include arithmetic, logical, and memory access operations. The instruction operation table below shows the basic instructions supported by the designed 4-stage pipelined processor.

Table 1.2.1: Basic Instruction Set of the 4-Stage Pipelined Processor

| **Opcode** | **Instruction** | **Operation Performed** | **Description** |
|---|---|---|---|
| 00 | ADD | Rdest = Rsrc1 + Rsrc2 | Adds two register values and stores the result in the destination register |
| 01 | SUB | Rdest = Rsrc1 − Rsrc2 | Subtracts the second register value from the first register |
| 10 | AND | Rdest = Rsrc1 & Rsrc2 | Performs bitwise AND operation between two registers |
| 11 | LOAD | Rdest = Memory[address] | Loads data from memory into the destination register |

## 1.3 Objective

The primary objective of this project is to design and simulate a Pipeline Processor using Verilog Hardware Description Language. The design is developed to demonstrate how instructions are executed through multiple pipeline stages, improving the performance of the processor.

The project aims to understand the working of pipelined processor architecture and its implementation using hardware description languages. Functional verification of the processor is carried out using a testbench and simulation waveform analysis to ensure correct instruction execution under different conditions.

Another objective is to perform synthesis of the design and analyze performance parameters such as area utilization, power consumption, and timing characteristics. The project provides practical exposure to processor design concepts and the VLSI design flow using Cadence simulation and synthesis tools.

## 1.4 Design Methodology

The Pipeline Processor is implemented using behavioral modeling in Verilog Hardware Description Language. The design includes instruction memory, register file, and pipeline registers used to store intermediate data between different pipeline stages.

The processor operates based on a clock signal, and instructions move through the pipeline stages during each clock cycle. Arithmetic and logical operations are performed depending on the instruction type, and the results are written back to the register file.

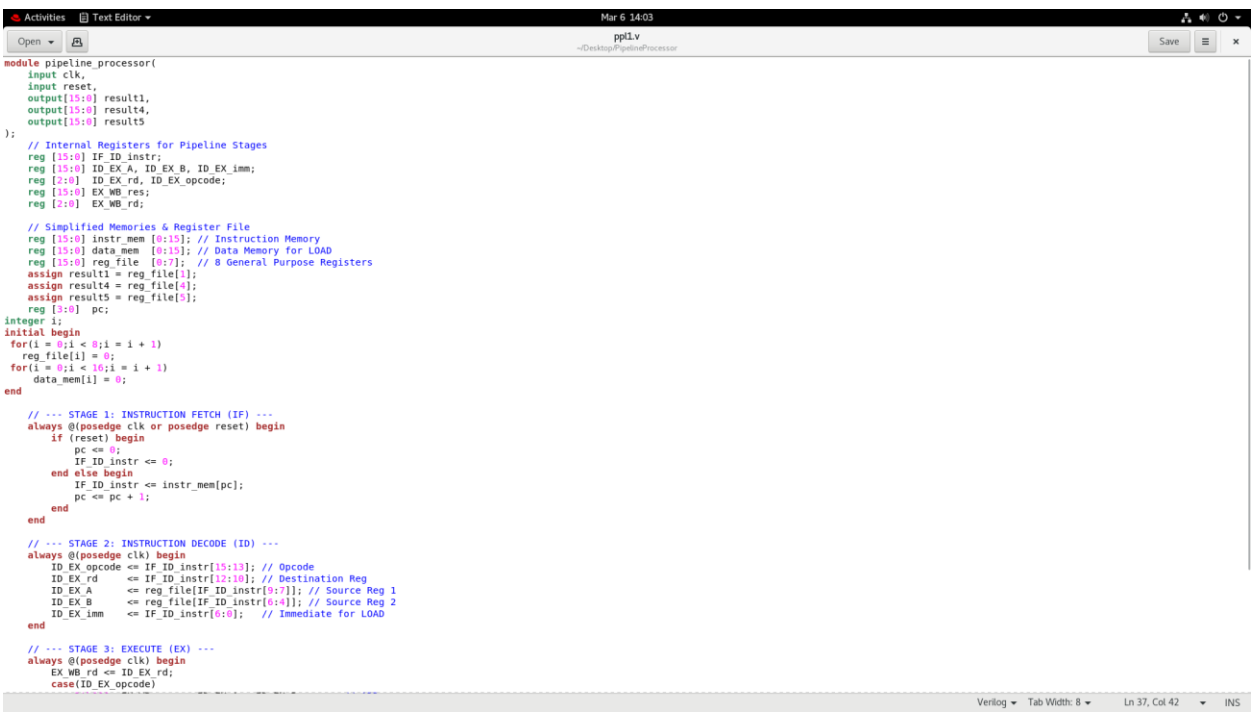
A testbench is developed to verify the functional behavior of the design by applying clock and reset signals and monitoring register outputs. The design is simulated using Cadence simulation tools, and waveform results are analyzed to confirm correct processor operation. After successful simulation, synthesis is performed to evaluate area, power, and timing performance.

# Chapter 2

# DESIGN METHODOLOGY

## 2.1 Pipeline Processor Design Implementation

The 4-stage pipelined processor was implemented using Verilog Hardware Description Language. The design consists of instruction memory, register file, arithmetic logic unit (ALU), and pipeline registers that transfer data between pipeline stages.



```
module pipeline_processor(
    input clk,
    input reset,
    output[15:0] result1,
    output[15:0] result4,
    output[15:0] result5
);
    // Internal Registers for Pipeline Stages
    reg [15:0] IF_ID_instr;
    reg [15:0] ID_EX_A, ID_EX_B, ID_EX_imm;
    reg [2:0]  ID_EX_rd, ID_EX_opcode;
    reg [15:0] EX_WB_res;
    reg [2:0]  EX_WB_rd;

    // Simplified Memories & Register File
    reg [15:0] instr_mem [0:15]; // Instruction Memory
    reg [15:0] data_mem  [0:15]; // Data Memory for LOAD
    reg [15:0] reg_file  [0:7];  // 8 General Purpose Registers
    assign result1 = reg_file[1];
    assign result4 = reg_file[4];
    assign result5 = reg_file[5];
    reg [3:0]  pc;
integer i;
initial begin
 for(i = 0;i < 8;i = i + 1)
   reg_file[i] = 0;
 for(i = 0;i < 16;i = i + 1)
     data_mem[i] = 0;
end

    // --- STAGE 1: INSTRUCTION FETCH (IF) ---
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            pc <= 0;
            IF_ID_instr <= 0;
        end else begin
            IF_ID_instr <= instr_mem[pc];
            pc <= pc + 1;
        end
    end

    // --- STAGE 2: INSTRUCTION DECODE (ID) ---
    always @(posedge clk) begin
        ID_EX_opcode <= IF_ID_instr[15:13]; // Opcode
        ID_EX_rd     <= IF_ID_instr[12:10]; // Destination Reg
        ID_EX_A      <= reg_file[IF_ID_instr[9:7]]; // Source Reg 1
        ID_EX_B      <= reg_file[IF_ID_instr[6:4]]; // Source Reg 2
        ID_EX_imm    <= IF_ID_instr[6:0];   // Immediate for LOAD
    end

    // --- STAGE 3: EXECUTE (EX) ---
    always @(posedge clk) begin
        EX_WB_rd <= ID_EX_rd;
        case(ID_EX_opcode)
```

Figure 2.1.1: Verilog Implementation of 4-Stage Pipelined Processor

Figure 2.1.1 shows the Verilog implementation of the 4-stage pipelined processor module. The processor executes instructions using four pipeline stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), and Write Back (WB). Each stage performs a specific function in the instruction execution process. The instruction

fetch-stage retrieves instructions from memory, the decode stage interprets the instruction and reads register operands, the execute stage performs arithmetic or logical operations, and the write-back stage stores the result into the register file. Pipeline registers are used between stages to store intermediate data and ensure synchronized instruction flow through the processor.

## 2.2 Testbench Implementation

A testbench was developed to verify the functionality of the 4-stage pipelined processor design. The testbench applies clock and reset signals and monitors processor outputs during simulation.

```
module tb_processor();
    reg clk, reset;
    wire[15:0] result1;
    wire[15:0] result4;
    wire[15:0] result5;
    pipeline_processor uut(clk, reset);
    integer i;
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
initial begin
    reset = 1;
    #10;
    reset = 0;

    // Initialize instruction memory
    for (i = 0; i < 16; i = i + 1)
        uut.instr_mem[i] = 16'b0;

    // Initialize data memory
    for (i = 0; i < 16; i = i + 1)
        uut.data_mem[i] = 16'b0;

    // Initialize register file values
    uut.reg_file[2] = 10; // R2 = 10
    uut.reg_file[3] = 50; // R3 = 50

    // -------- Instructions --------

    // 🟦 LOAD R1, 5
    uut.instr_mem[0] = 16'b010_001_000_0000101;
    uut.data_mem[5] = 16'd50;

    // 🟦 ADD R4 = R2 + R3
    uut.instr_mem[1] = 16'b000_100_010_0110000;

    // 🟦 SUB R5 = R2 - R3
    uut.instr_mem[2] = 16'b001_101_010_0110000;

    #200;
    $finish;
end
endmodule
```

Figure 2.2.1: Testbench Implementation of 4-Stage Pipelined Processor

Figure 2.2.1 shows the Verilog testbench used to verify the operation of the pipelined processor. The testbench generates the clock signal required for pipeline stage transitions and initializes the reset signal to start the processor operation. It applies different instruction inputs and monitors the outputs produced by the processor

during simulation. The testbench ensures that instructions move correctly through each pipeline stage and that the processor produces the expected results.

## 2.3 RTL / Schematic View

After synthesis, the RTL schematic of the processor is generated to visualize the hardware structure of the design.

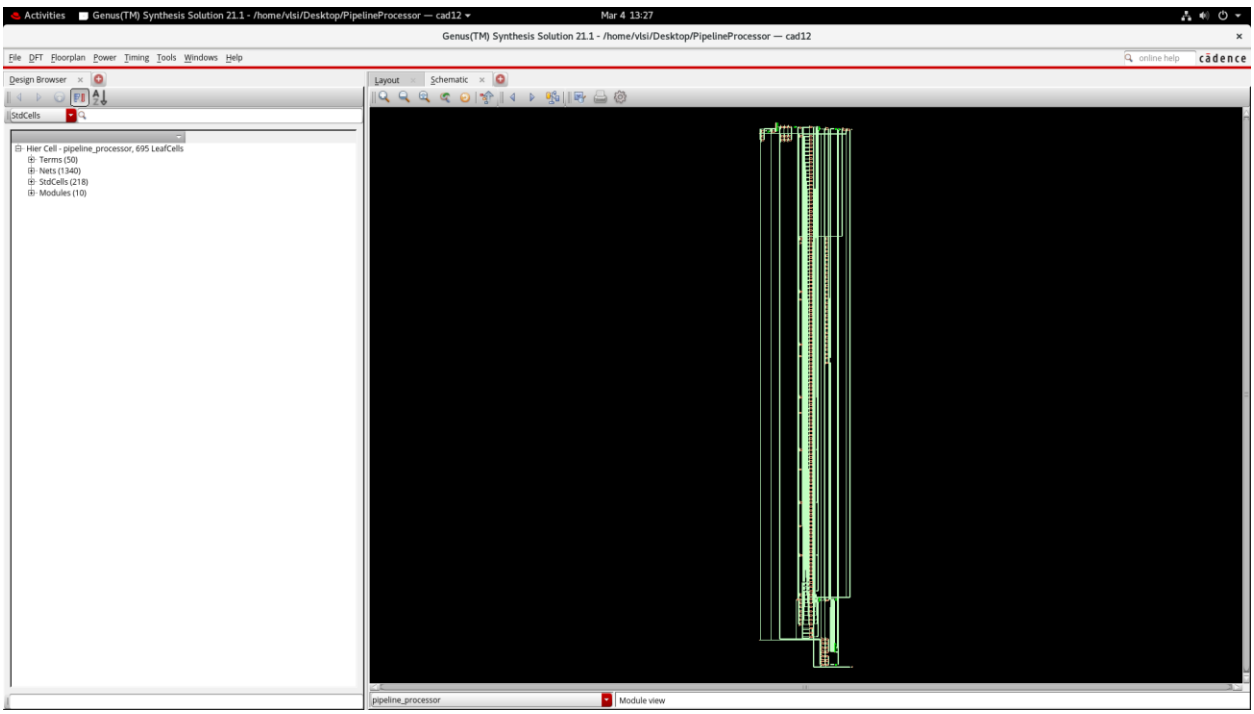



Figure 2.3.1: Synthesized RTL Schematic of 4-Stage Pipelined Processor

Figure 2.3.1 shows the synthesized RTL schematic of the pipelined processor generated using Cadence tools. The schematic represents the internal hardware architecture of the processor, including instruction memory, registers, control logic, and pipeline registers connecting the different pipeline stages. This schematic verifies that the behavioral Verilog code has been successfully converted into a hardware-level implementation suitable for digital system design.

# Chapter 3

# Simulation

## 3.1 Simulation Methodology

The 4-stage pipelined processor design and its testbench were simulated using Cadence simulation tools. The Verilog design files were compiled and executed to verify the functional behavior of the processor. The simulation was performed to ensure that instructions move correctly through the pipeline stages during each clock cycle.

The simulation waveforms were generated using the SimVision waveform viewer. Important signals such as the clock signal, reset signal, program counter, pipeline registers, instruction signals, and execution results were observed during the simulation process. Different instruction operations including arithmetic and logical operations were verified by monitoring the corresponding output values. The results obtained from the waveform analysis were compared with expected theoretical behavior to confirm correct operation of the pipelined processor.

## 3.2 Simulation Results and Analysis

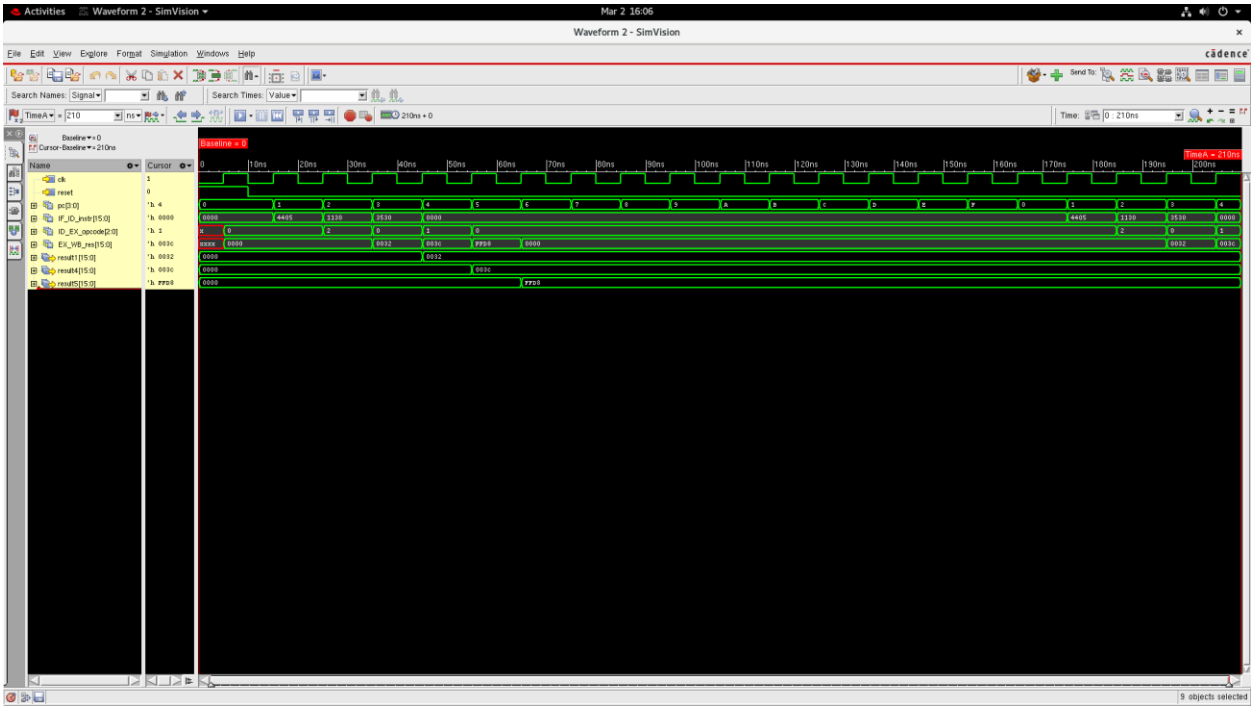



Figure 3.2.1: Simulation Waveform of 4-Stage Pipelined Processor

The simulation results of the 4-stage pipelined processor were verified using the SimVision waveform viewer, as shown in Figure 3.2.1. The waveform displays important processor signals including the clock signal, reset signal, program counter (PC), instruction registers, pipeline stage signals, and execution results.

During simulation, the clock signal drives the operation of the pipeline stages and the program counter increments sequentially to fetch instructions from instruction memory. The fetched instruction is then passed to the instruction decode stage where it is interpreted and the required operands are identified.

The instruction then moves to the execution stage where arithmetic or logical operations such as ADD, SUB, AND, and LOAD are performed. The results generated during the execution stage are passed to the write-back stage where they are stored in the register file.

Intermediate pipeline registers such as IF/ID, ID/EX, and EX/WB are used to transfer data between pipeline stages. These registers ensure that instructions move correctly through the pipeline during each clock cycle. The observed waveform confirms that instructions progress sequentially through the pipeline stages and the processor produces the correct results for the executed instructions.

The simulation results match the expected behavior of the designed processor, confirming correct pipeline operation and functional accuracy of the 4-stage pipelined processor.

# Chapter 4

# Synthesis And Performance Analysis

After successful simulation, the 4-stage pipelined processor design was synthesized using Cadence Genus to evaluate the hardware performance parameters of the processor implementation. Synthesis converts the behavioral Verilog description into a gate-level hardware representation suitable for physical implementation. Performance parameters such as power consumption, area utilization, and timing characteristics were analyzed using synthesis reports.

## 4.1 Power Analysis

The power report provides information about the dynamic and leakage power consumption of the pipelined processor design.

```
Instance: /pipeline_processor
Power Unit: W
PDB Frames: /stim#0/frame#0
  ----------------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ----------------------------------------------------------------------------
      memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
    register     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
       latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
       logic     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
        bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
       clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
         pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
          pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 100.00%
  ----------------------------------------------------------------------------
    Subtotal     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00 800.00%
  Percentage         100.00%      100.00%      100.00%      100.00% 100.00%
  ----------------------------------------------------------------------------
```

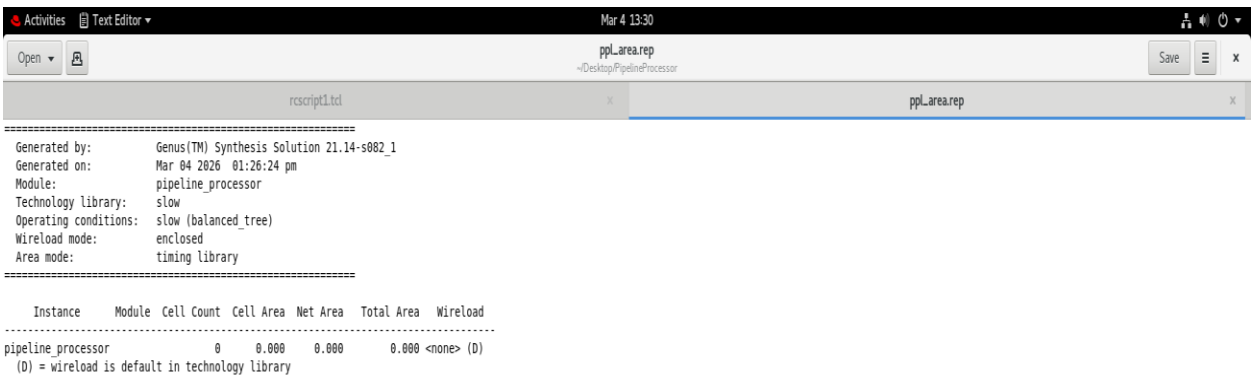
Figure 4.1.1: Power Analysis Report

Figure 4.1.1 shows the power analysis report of the synthesized 4-stage pipelined processor generated using Cadence Genus. The report provides details about the total power consumed by the processor during operation. It includes dynamic power caused by switching activity of the logic gates and leakage power due to static current in the circuit. Power analysis is an important parameter in VLSI design

because it determines the energy efficiency of the system. The results indicate that the processor operates within acceptable power limits, demonstrating an efficient hardware implementation.

## 4.2 Area Analysis

The area report shows the number of logic cells and hardware resources used in the processor implementation.



```
Activities   Text Editor   Mar 4 13:30
Open   ppl_area.rep   Save
~/Desktop/PipelineProcessor
rcscript1.tcl   ppl_area.rep
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Mar 04 2026  01:26:24 pm
  Module:                 pipeline_processor
  Technology library:     slow
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

    Instance     Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------------
pipeline_processor                 0      0.000     0.000       0.000 <none> (D)
  (D) = wireload is default in technology library
```

Figure 4.2.1: Area Utilization Report

Figure 4.2.1 presents the area utilization report of the synthesized pipelined processor design. The report shows the total number of standard cells and logic elements used along with the overall hardware area occupied by the processor circuit. Area analysis is an important factor in VLSI design because it affects chip size, fabrication cost, and hardware complexity. The synthesis results indicate that the pipelined processor design uses hardware resources efficiently while maintaining correct functionality.

## 4.3 Timing Analysis

Timing analysis evaluates the speed and delay characteristics of the processor circuit.

```
Activities   Text Editor   Mar 4 13:30
Open   ppl_timing.rep   ~/Desktop/PipelineProcessor   Save
rcscript1.tcl   ppl_area.rep   ppl_gate.rep   ppl_netlist.v   ppl_power.rep   ppl_timing.rep
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Mar 04 2026  01:26:24 pm
  Module:                 pipeline_processor
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

No paths found
```

Figure 4.3.1: Timing Analysis Report

Figure 4.3.1 shows the timing analysis report of the synthesized pipelined processor design. The timing report provides information about propagation delay, critical path delay, and timing constraints of the circuit. Timing analysis ensures that the processor operates correctly within the specified clock period without timing violations. The results confirm that the processor meets the required timing constraints and operates reliably.

## 4.4 Gate-Level Report

The gate-level synthesis report shows the number and types of logic gates used in the processor implementation.

```
Activities   Text Editor ▾                    Mar 4 13:30
Open ▾                    ppl_gate.rep                                   Save
                          ~/Desktop/PipelineProcessor
rcscript1.tcl            ppl_area.rep            ppl_gate.rep
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Mar 04 2026  01:26:24 pm
  Module:                 pipeline_processor
  Technology library:     slow
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================
```

Figure 4.4.1: Gate-Level Synthesis Report

Figure 4.4.1 shows the gate-level synthesis report generated after synthesizing the 4-stage pipelined processor design. The report provides information about the different types of logic gates used in the hardware implementation of the processor. Gate-level analysis helps in understanding the hardware complexity and structural efficiency of the design. The results indicate that the processor is implemented using an optimized number of logic gates, ensuring efficient hardware realization.

# Chapter 5

# Results

The simulation results confirm the correct functional operation of the designed 4-stage pipelined processor for all tested conditions. During simulation, instructions successfully passed through the pipeline stages including Instruction Fetch, Instruction Decode, Execute, and Write Back. The waveform analysis clearly demonstrated that instructions were fetched from instruction memory, decoded correctly, executed by the arithmetic logic unit, and the final results were written back to the register file.

The processor correctly performed the basic instructions supported by the design such as ADD, SUB, AND, and LOAD. The simulation waveform verified that arithmetic and logical operations produced the expected output values during the execution stage and that the results were properly stored in the register file during the write-back stage. The program counter was observed to increment sequentially, confirming proper instruction fetching and pipeline progression.

Furthermore, synthesis analysis provided important information about the hardware performance of the processor design. The area report indicated efficient utilization of logic cells and hardware resources, demonstrating an optimized hardware implementation of the processor architecture. The power analysis showed acceptable dynamic and leakage power consumption, indicating energy-efficient processor operation. Timing analysis confirmed that the processor meets the required timing constraints without any violations, ensuring reliable and stable operation.

Overall, the obtained results validate both the functional correctness and hardware efficiency of the designed 4-stage pipelined processor.

# Chapter 6

# Discussion

This project demonstrates the implementation of a 4-stage pipelined processor using Verilog Hardware Description Language. The processor architecture was designed and verified through simulation using Cadence tools. The design shows how instructions are processed through multiple pipeline stages to improve processor performance.

The pipeline stages including Instruction Fetch, Instruction Decode, Execute, and Write Back allow multiple instructions to be processed simultaneously. Pipeline registers between stages ensure proper synchronization and smooth movement of instructions through the processor.

Simulation results confirmed that the processor correctly executes the supported instructions such as ADD, SUB, AND, and LOAD. The waveform analysis verified that instructions pass sequentially through each pipeline stage and produce the correct results.

The synthesis reports further confirmed that the processor design operates efficiently in terms of area, power consumption, and timing performance. Overall, the project demonstrates the practical implementation of pipelined processor architecture and the VLSI design flow using Verilog and Cadence tools.

# Chapter 7

# Conclusion

The 4-stage pipelined processor was successfully designed, implemented, simulated, and synthesized using Verilog Hardware Description Language and Cadence tools. The processor architecture includes four pipeline stages: Instruction Fetch, Instruction Decode, Execute, and Write Back, which enable efficient instruction processing.

Functional verification through simulation confirmed that the processor correctly executes the supported instructions such as ADD, SUB, AND, and LOAD. The simulation waveform demonstrated that instructions move sequentially through the pipeline stages and produce the expected results.

The synthesis analysis showed efficient utilization of hardware resources with acceptable area, power consumption, and timing performance. The design successfully meets the required functionality and performance requirements.

Overall, this project provided practical experience in pipelined processor design, simulation, and synthesis, and demonstrated the implementation of a basic processor architecture using the VLSI design flow.